Tutoriel : STELLAR WINGS en C

Écrire un shoot'em up GX4000 en C avec GX Studio, puis l'accélérer en assembleur

GX Studio 0.5.1 — SDCC — sprites matériels, rasters, son DMA



Le résultat : STELLAR WINGS C sur GX4000

Ce tutoriel construit STELLAR WINGS en C, en quatre étapes. Chaque étape se compile et se joue. Le projet de chaque étape est fourni dans le dossier **tutoriel/** du projet (etape1, etape2, etape3) ; l'étape 4 est le projet final. On peut les ouvrir dans le studio pour comparer avec son propre travail, ou partir de l'un d'eux.

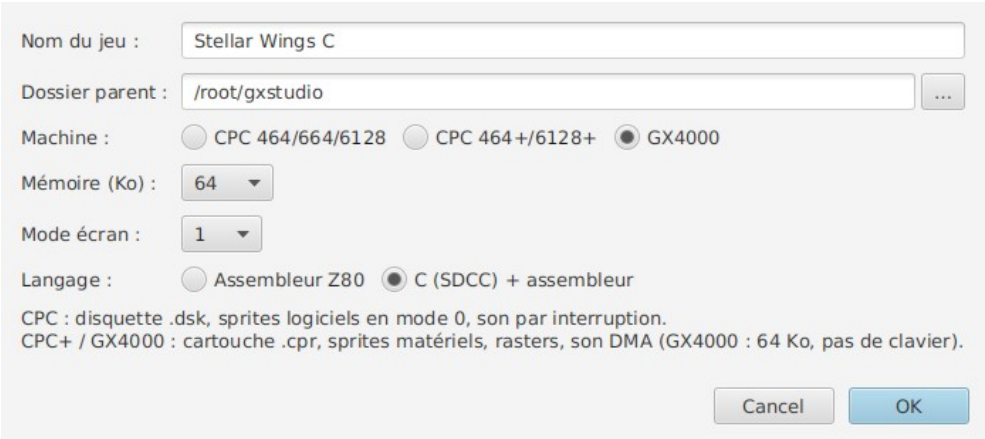
Étape	Ce que l'on fait	Ce que l'on apprend
0	Créer le projet en C, les ressources	projet C, sprites, palettes, sons, images
1	Écran titre, vaisseau, tirs, étoiles	structure d'un jeu en C, gx.h, boucle à 50 images/s
2	La formation de 24 aliens avec 6 sprites	interruptions raster, multiplexage, double tampon, collisions
3	Le jeu complet : plongeurs, bombes, capsules, vaisseau mère, joueurs	machine à états, organisation en fichiers
4	Mesurer, puis passer deux routines en assembleur	profil, appel de l'assembleur depuis le C

Il faut GX Studio 0.5.1 réglé (rasm, Caprice32 corrigé 0.5.1) et le compilateur C **SDCC** : `sudo apt install sdcc`. Le tutoriel du casse-brique (chapitre 12) est une bonne première approche du C dans le studio. La documentation du code (Stellar_Wings_C_documentation.pdf) détaille chaque fonction.

0. Le projet et les ressources

0.1 Créer le projet

Fichier > Nouveau projet : nom **Stellar Wings C**, machine **GX4000** (64 Ko), mode écran **1**, langage **C (SDCC) + assembleur**.

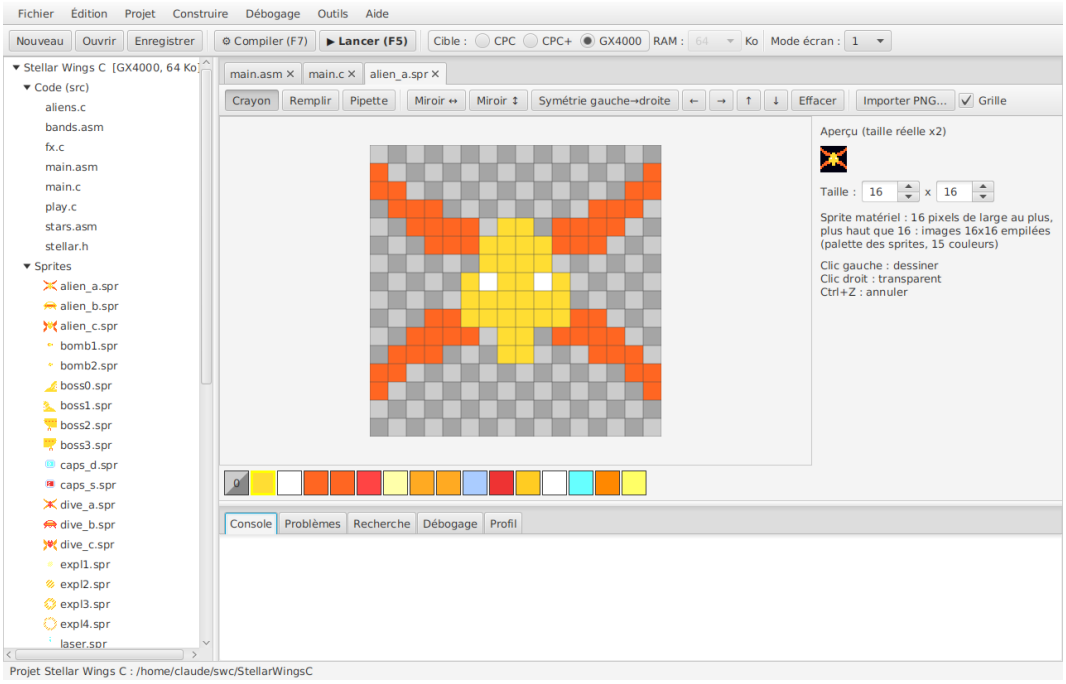


Un projet en C pour la GX4000

Le studio crée src/main.c (un petit jeu de démonstration), src/main.asm (qui assemble la cartouche) et la bibliothèque lib/. On garde main.asm tel quel jusqu'à l'étape 4. On supprime les sprites et les sons de démonstration (clic droit > Supprimer) ; on garde les deux palettes, qu'on modifie.

0.2 Les sprites

Le jeu utilise 20 images de 16 x 16 pixels. On les dessine avec **Projet > Nouveau sprite…**, ou on recopie le dossier assets/sprites de tutoriel/etape1. Les noms comptent. Le studio numérote les images par ordre alphabétique (F_ALIEN_A, F_ALIEN_B…), et le programme calcule des numéros comme `F_ALIEN_A + type` : les images d'une même famille doivent avoir des noms qui se suivent.



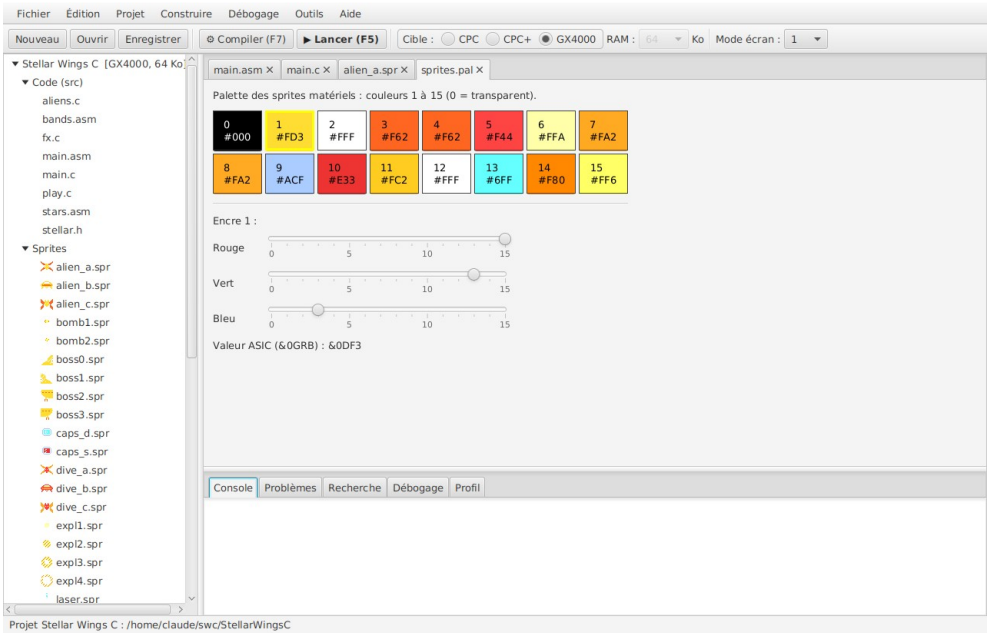
alien_a.spr : les couleurs 1 à 4 seulement (elles changeront à chaque rangée)

Fichiers	Contenu

alien_a, alien_b, alien_c	les trois formes d'aliens, en couleurs 1 à 4 (3 = ailes hautes, 4 = ailes basses)
dive_a, dive_b, dive_c	les mêmes en couleurs 5 à 8 (plongeurs)
ship, laser	vaisseau, tir
bomb1, bomb2 ; expl1 à expl4	bombe (2 images), explosion (4 images)
caps_d, caps_s	capsules D et S
boss0 à boss3	les 4 quarts du vaisseau mère

0.3 Les palettes

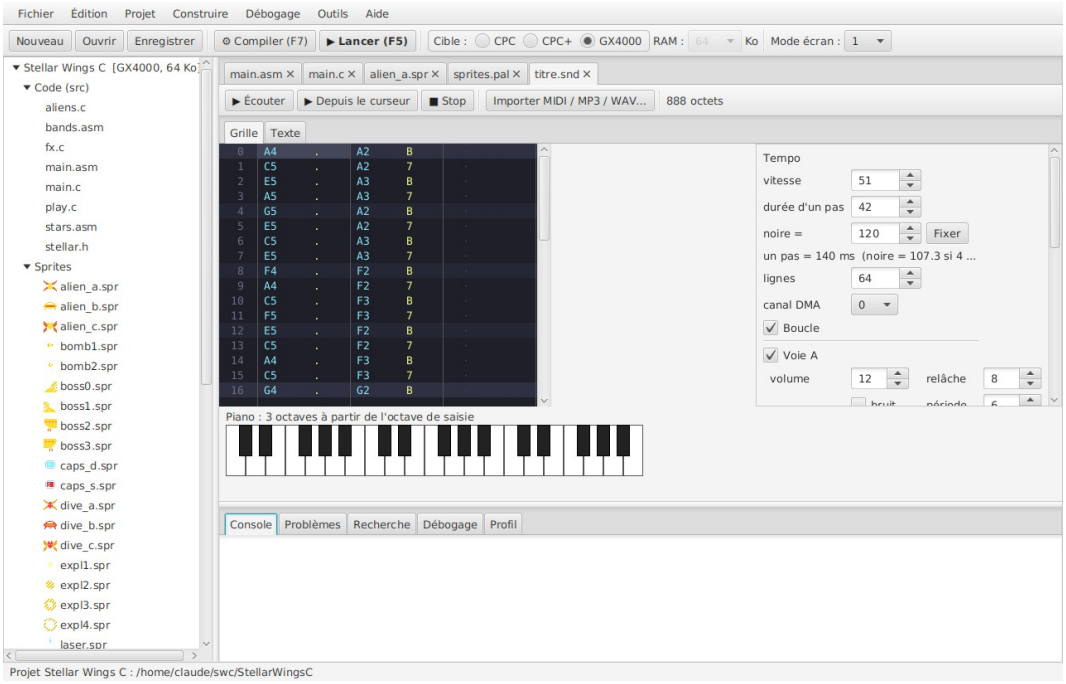
ecran.pal : encres 0 à 3 du jeu (0 = fond bleu nuit #001, 1 à 3 = étoiles et textes, du plus sombre au plus clair). **sprites.pal** : couleurs 1 à 15 des sprites. La couleur 0 d'un sprite est toujours transparente.



sprites.pal

0.4 Les sons

Dix fichiers .snd. La musique du titre est écrite en notes : une ligne par voie, A pour la mélodie et B pour la basse. Les bruitages sont des suites d'instructions (TONE, VOL, NOISE, PAUSE). On peut les écouter dans l'éditeur sans lancer l'émulateur.



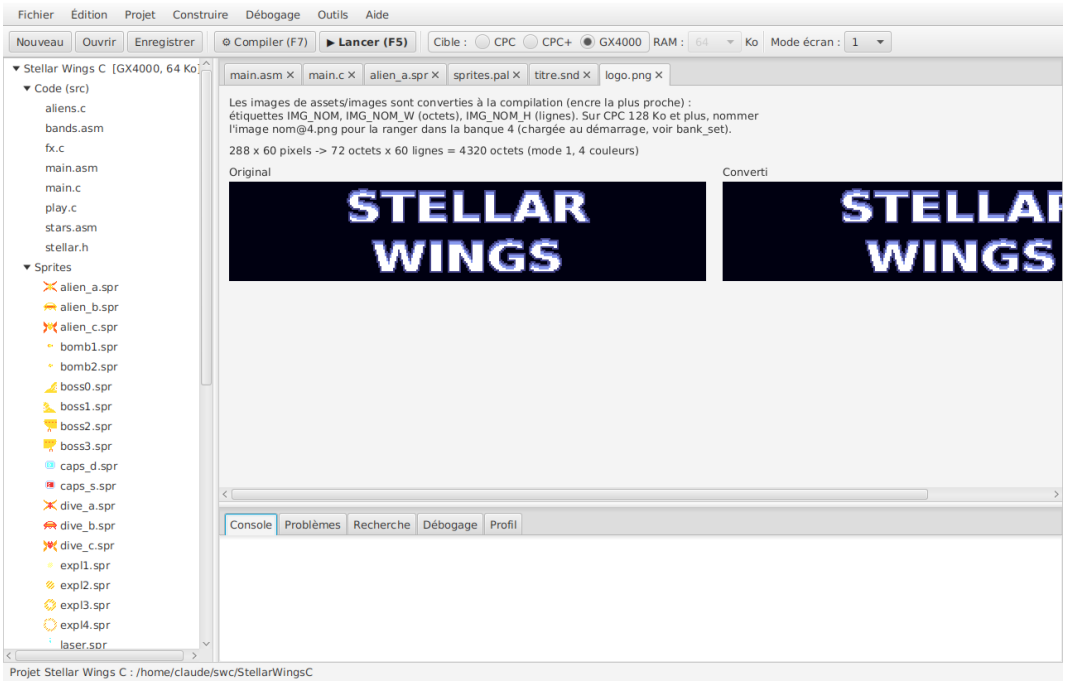
titre.snd dans l'éditeur de sons

```
; tir du joueur : note qui descend, voie B
canal 1
vitesse 0
TONE B 50
VOL B 13
PAUSE 156
TONE B 80
VOL B 12
PAUSE 156
TONE B 110
VOL B 11
PAUSE 156
TONE B 140
VOL B 10
PAUSE 156
TONE B 170
VOL B 9
PAUSE 156
TONE B 200
VOL B 8
PAUSE 156
TONE B 230
VOL B 7
PAUSE 156
TONE B 260
VOL B 6
PAUSE 156
TONE B 290
VOL B 5
PAUSE 156
VOL B 0
```

Fichier	canal / vitesse	Rôle
titre, vague, bravo, gameover	0 / 51	musique et jingles
marche	0 / variable	marche de la formation (le programme change son tempo)
tir, capsule, boss_touche	1 / 0	bruitages, voie B
boom, mort	2 / 0	explosions, voie C en bruit

0.5 Le logo

Une image PNG dans **assets/images** est convertie dans le mode de l'écran. Le studio produit IMG_LOGO (les octets), IMG_LOGO_W (largeur en octets) et IMG_LOGO_H. logo.png fait 288 x 60 pixels, en 4 couleurs prises dans ecran.pal.



logo.png et sa conversion en mode 1

1. Écran titre, vaisseau, tirs, étoiles

Projet de cette étape : **tutoriel/etape1**. On écrit quatre fichiers : stellar.h (déclarations communes), main.c (titre et partie), play.c (une image de jeu), fx.c (hasard, score, étoiles). On supprime le main.c de démonstration.

1.1 stellar.h

Un en-tête commun évite de répéter les constantes et les déclarations. Il inclut gx.h, qui donne accès à la bibliothèque et aux ressources (build/gen.h). Les sprites ont chacun un rôle fixe : 0 à 5 pour la formation, 6 pour le vaisseau, 7 et 8 pour les tirs… Les types regroupent ce qui va ensemble : un joueur, un tir, un plongeur.

```
/*---- sprites materiels ------------------------------------------------
 *  0-5   formation (multiplexee : 4 rangees par interruptions raster) ou boss (0-3)
 *  6     vaisseau        7-8  tirs du joueur     9-10 bombes
 *  11    capsule         12-14 plongeurs         15   explosion          */
#define SPR_SHIP    6
#define SPR_BULLET  7
#define SPR_BOMB    9
#define SPR_CAPS    11
#define SPR_DIVER   12
#define SPR_EXPL    15

/*---- types -----------------------------------------------------------------*/
typedef struct {                /* un joueur (deux blocs : partie a deux en alternance) */
    uint16_t score;             /* en dizaines de points (affiche avec un 0 de plus) */
    uint8_t  lives;
    uint8_t  wave;              /* numero de vague (1, 2...) */
    uint8_t  alive[4];          /* aliens vivants : rangee r, bit c = colonne c */
    uint8_t  power;             /* bit 0 : tir double (capsule D), bit 1 : vitesse (S) */
    uint8_t  over;              /* 1 = partie terminee pour ce joueur */
    uint8_t  bhp;               /* points de vie du vaisseau mere */
} player_t;

typedef struct {                /* tir du joueur ou bombe */
```

```
    uint8_t on;
    int16_t x;                  /* pixels mode 2 */
    uint8_t y;
} shot_t;

typedef struct {                /* alien en pique */
    uint8_t state;              /* 0 libre, 1 decollage, 2 attaque, 3 retour */
    uint8_t row, col;           /* sa place dans la formation */
    int16_t x, y;               /* y signe : il repasse par le haut de l'ecran */
    int8_t  vx;
    uint8_t timer, bomb;
} diver_t;
```

Les variables partagées sont déclarées `extern` dans l'en-tête et définies une seule fois dans un fichier .c. Le fichier complet déclare déjà toutes les fonctions du jeu final : celles qui ne sont pas encore écrites ne gênent pas tant qu'on ne les appelle pas.

1.2 Le programme principal

`main()` est appelée par src/main.asm quand la console est prête. `gx_init()` prépare le son et les sprites. Ensuite l'écran titre et la partie s'enchaînent sans fin.

```
static void title_screen(void)
{
    screen_reset();
    gx_blit(IMG_LOGO, IMG_LOGO_W, IMG_LOGO_H, (80 - IMG_LOGO_W) / 2, LOGO_Y);
    gx_ink(1);
    msg_center("- L'ATTAQUE DES FAUCONS -", 84);
    gx_ink(2);
    msg_center("TIR 1 : UN JOUEUR", 128);
    gx_ink(3);
    gx_text(13, 162, "RECORD");
    print_score(21, 162, hiscore);
    gx_sprite_palette(pal_sprites);
    gx_palette(pal_title);
    GX_SOUND(TITRE);
    do {
        gx_wait_frame();
        gx_spr_update_from(0);
        title_parade();
        rand8();
        gx_read_joy();
    } while (!(joy_new & JOY_FIRE));
    nplayers = 1;
}

/* game : pour l'instant, le vaisseau seul sous les etoiles */
static void game(void)
{
    ply.lives = 3;
    ply.wave = 1;
    screen_reset();
```

- `gx_blit` copie le logo dans l'écran. Le logo est dans la page 2 de la cartouche, lue en &C000 ; les écritures à ces adresses vont dans l'écran, qui est en RAM.
- `GX_SOUND(TITRE)` lance la liste DMA du fichier titre.snd sur son canal : la musique joue toute seule.
- La boucle attend la fin de l'image (`gx_wait_frame`), recopie les sprites dans l'ASIC (`gx_spr_update_from(0)`), anime, lit les manettes. C'est le schéma de toute boucle de jeu.

```
static void game(void)
{
    ply.lives = 3;
    ply.wave = 1;
    screen_reset();
    gx_palette(pal_screen);
    gx_sprite_palette(pal_sprites);
    stars_init();
    hud_draw();
    turn_reset();
```

```
    for (;;) play_frame();
}
```

1.3 Une image de jeu

`play_frame` est appelée 50 fois par seconde. Elle commence toujours par `gx_wait_frame()` : la bibliothèque a vu passer la ligne 199, l'écran visible est fini, c'est le moment de recopier les sprites. Ensuite on lit les manettes et on fait avancer chaque objet.

```
void play_frame(void)
{
    gx_wait_frame();                    /* ligne 199 : bord bas, rien n'est affiche */
    gx_spr_update_from(SPR_SHIP);       /* sprites 6-15 */
    gx_read_joy();
    stars_update();
    ship_update();
    bullets_update();
    if (hud_dirty) hud_update();
}
```

```
static void ship_update(void)
{
    if (joy & JOY_LEFT) {
        ship_x -= 4;
        if (ship_x < 8) ship_x = 8;
    }
    if (joy & JOY_RIGHT) {
        ship_x += 4;
        if (ship_x > 600) ship_x = 600;
    }
    if (fire_cool) {
        fire_cool--;
    } else if ((joy & JOY_FIRE) && !bullets[0].on) {
        fire_bullet(&bullets[0], 0);    /* bouton maintenu = tir automatique */
        fire_cool = 10;
        GX_SOUND(TIR);
    }
    gx_spr_set(SPR_SHIP, ship_x, SHIP_Y, F_SHIP, ZOOM_X2Y1);
}
```

`joy` contient l'état des manettes après `gx_read_joy()` : un bit par direction et par bouton (JOY_LEFT, JOY_FIRE…). Les sprites se placent en pixels « mode 2 », de 0 à 639 ; le zoom x2 en largeur (ZOOM_X2Y1) double leur taille. `gx_spr_set` n'écrit que dans une copie en RAM : l'ASIC est mis à jour par gx_spr_update_from au début de l'image suivante. Les déplacements ne se voient donc jamais à moitié.

1.4 Les étoiles

40 étoiles sont dessinées directement dans la mémoire écran. `GX_LINE(y)` donne l'adresse de la ligne y : les lignes du CPC ne se suivent pas en mémoire, la table line_addr générée par le studio fait le calcul. Un octet du mode 1 contient 4 pixels : le motif (pat_tab) allume un pixel dans l'encre voulue. Les étoiles rapides ont l'encre la plus claire.

```
void stars_update(void) __sdcccall(0)
{
    uint8_t i, y;
    uint8_t *p;
    star_t *s = stars;
    for (i = NSTARS; i; i--, s++) {
        p = star_ptr(s->y, s->col);
        if (p) *p = 0;                              /* efface */
        y = s->y + s->speed;
        if (y >= 184) {                             /* repart en haut, colonne au hasard */
            y -= 168;
            s->col = star_col();
        }
        s->y = y;
        p = star_ptr(y, s->col);
        if (p) *p = s->pat;                         /* dessine */
```

```
    }
}
```

F5 : après les 3 secondes de l'écran de présentation de GX Studio (affiché par la bibliothèque au démarrage de tout jeu), le titre apparaît, puis le vaisseau qui tire sous les étoiles.



Étape 1 : écran titre et vaisseau

2. La formation : 24 aliens avec 6 sprites

Projet de cette étape : **tutoriel/etape2**. La GX4000 n'a que 16 sprites, et la formation compte 24 aliens. L'astuce : les sprites 0 à 5 montrent la rangée 0 ; juste après, une **interruption raster** les déplace sur la rangée 1, et ainsi de suite. C'est le **multiplexage**. On ajoute le fichier **aliens.c**.

2.1 Les aliens vivants

Chaque rangée tient dans un octet : bit c = l'alien de la colonne c est vivant. Quatre octets décrivent donc toute la formation (`ply.alive[4]`). Détruire un alien, c'est effacer un bit ; les compter, c'est compter les bits.

```
void count_alive(void)
{
    uint8_t r, a, n = 0, cols = 0;
    brow = 0;
    for (r = 0; r < 4; r++) {
        a = ply.alive[r];
        if (a) {
            cols |= a;
            brow = r;
            for (; a; a >>= 1) n += a & 1;
        }
    }
    alive_count = n;
    if (!cols) return;
    for (lcol = 0; !(cols & bits[lcol]); lcol++) ;
    for (rcol = 5; !(cols & bits[rcol]); rcol--) ;
}
```

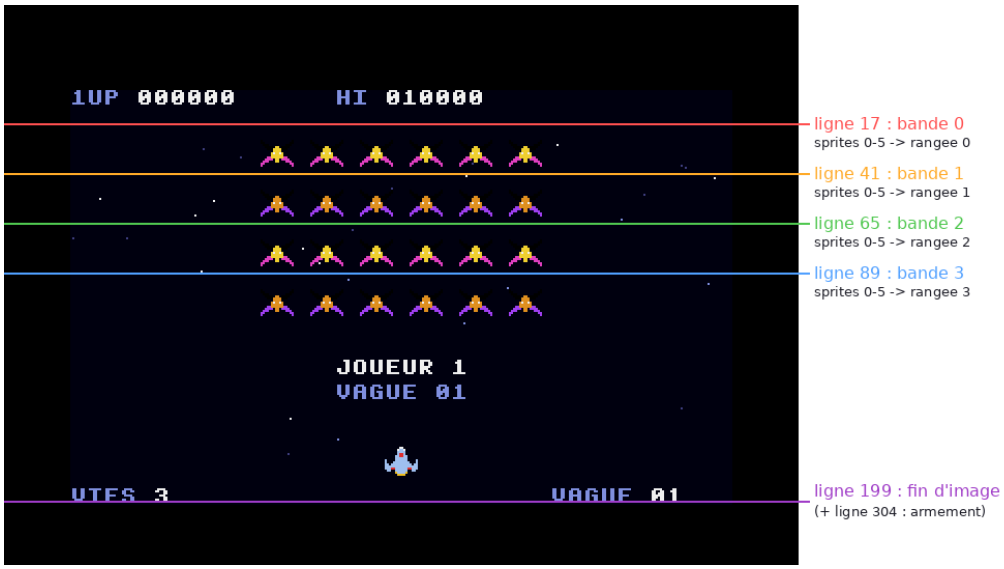
2.2 Le mouvement

La formation avance en quarts de pixel (`fsub` garde la fraction) : sa vitesse peut ainsi augmenter très progressivement quand des aliens disparaissent. Au bord, elle repart dans l'autre sens et descend de 4 lignes.

```
void form_update(void)
{
    uint8_t v, s, edge;
    int16_t ex;
    wing = (frame_count >> 3) & 1;
    if (march_on) {
        v = march_tempo_val();
        if (v != last_tempo) {
            last_tempo = v;
            gx_sound_tempo(v);
        }
    }
    if (frozen || !alive_count) return;
    v = ((28 - alive_count) >> 2) + 3;          /* vitesse en 1/4 de pixel par image */
    s = ply.wave >> 1;
    v += s < 7 ? s : 6;
    if (alive_count == 1) v += 8;               /* le dernier alien fonce */
    s = fsub + v;
    fsub = s & 3;
    v = s >> 2;
    if (!fdir) {
        fx += v;
        ex = fx + rcol * COL_PITCH;             /* X de la colonne la plus a droite */
        if (ex < 640 - 16 - 32) return;
    } else {
        fx -= v;
        ex = fx + lcol * COL_PITCH;             /* X de la colonne la plus a gauche */
        if (ex >= 16) return;
    }
    fdir ^= 1;                                  /* demi-tour... */
    if (fy < 148) fy += 4;                      /* ...et la formation descend d'un cran */
    edge = brow * ROW_PITCH + fy + 16;
    if (edge >= 164) kill_ship();               /* invasion : la formation atteint le
vaisseau */
}
```

2.3 La table raster

Une **table raster** commence par la ligne de la première interruption. Elle contient ensuite des entrées de 4 octets : type, donnée (un mot), ligne suivante. Le type 1 « bande de sprites » donne l'adresse d'un bloc de 38 octets. L'interruption le recopie dans l'ASIC : X, Y et zoom des sprites 0 à 5, puis leurs couleurs 1 à 4. On place l'entrée 7 lignes avant la rangée, pour laisser au Z80 le temps de recopier. La table se termine par la fin d'image (type 2) et l'armement (type 3).



Une bande par rangée : les mêmes 6 sprites, 4 fois par image

```
void build_raster(void)
{
    uint8_t *t = next_tab(), *b = br_blk;
    uint8_t r, y = fy;
    *t++ = fy - 7;                              /* 1re interruption : 7 lignes avant la
rangee 0 */
    for (r = 0; r < 4; r++) {
        *t++ = 1;                               /* entree : bande de sprites */
        *t++ = (uint8_t)(uint16_t)b;
        *t++ = (uint16_t)b >> 8;
        *t++ = r == 3 ? 199 : y + ROW_PITCH - 7;
        band_row(b, fx, y, ply.alive[r] & ~diving[r], rowpal + r * 4, wing);
        b += GX_BAND_SIZE;
        y += ROW_PITCH;
    }
    tab_tail(t);
}
```

```
void band_row(uint8_t *b, int16_t x, uint8_t y, uint8_t vis,
              const uint16_t *pal, uint8_t wg) __sdcccall(0)
{
    uint8_t i;
    for (i = 0; i < 6; i++) {
        b = put_spr(b, x, y, (vis & 1) ? ZOOM_X2Y1 : 0);
        vis >>= 1;
        x += COL_PITCH;
    }
    put_colors(b, pal);
    if (wg) b[4] = b[5] = 0;                    /* ailes basses : couleur 3 au noir */
    else    b[6] = b[7] = 0;                    /* ailes hautes : couleur 4 au noir */
}
```

Les positions changent à chaque image, mais l'interruption lit la table pendant l'affichage. On écrit donc dans une **autre** table : `next_tab()` alterne entre deux tables. `gx_raster_queue()` dit à la bibliothèque de passer à la nouvelle table à la fin de l'image. Les couleurs de chaque rangée viennent de rowpal_tab. Pour le battement d'ailes, une des couleurs 3 ou 4 passe au noir une image sur deux : les ailes semblent monter et descendre sans changer d'image.

```
uint8_t *next_tab(void)
{
    rbuf ^= 1;
    cur_tab = rtab[rbuf];
    br_blk = bands[rbuf];
    return cur_tab;
}

/* tab_tail : fin d'image + armement, puis publication de la table */
void tab_tail(uint8_t *p)
{
```

```
    p[0] = 2; p[1] = 0; p[2] = 0; p[3] = RASTER_ARM_LINE;
    p[4] = 3; p[5] = 0; p[6] = 0; p[7] = 0;
    gx_raster_queue(cur_tab);                   /* prise en compte a la fin de l'image */
}
```

2.4 Toucher un alien

La pointe du tir est convertie en colonne (48 pixels par colonne) et en rangée (24 lignes par rangée). On utilise des soustractions répétées, pas `/` : sur le Z80, une division appelle une routine de SDCC bien plus lente. Il faut aussi que le tir tombe dans la partie dessinée de l'alien.

```
uint8_t hit_formation(int16_t hx, uint8_t hy)
{
    int16_t dx = hx - fx;
    uint8_t c, r, rx, ry;
    if (dx < 0 || dx >= 512) return 0;
    for (c = 0; dx >= COL_PITCH; c++) dx -= COL_PITCH;  /* colonne (division sans routine de
division) */
    rx = (uint8_t)dx;
    if (c >= 6 || rx < 4 || rx >= 28) return 0; /* entre deux colonnes */
    if (hy < fy) return 0;
    ry = hy - fy;
    for (r = 0; ry >= ROW_PITCH; r++) ry -= ROW_PITCH;  /* rangee */
    if (ry >= 16 || r >= 4) return 0;
    if (!alien_visible(r, c)) return 0;
    alien_destroy(r, c);
    add_score(row_pts[r]);
    expl_start(fx + c * COL_PITCH, fy + r * ROW_PITCH);  /* explosion a la place de l'alien
*/
    return 1;
}
```

F5 : la formation se balance et descend ; chaque rangée a ses couleurs ; les aliens explosent et le score monte. Quand la formation est détruite, BRAVO et vague suivante (main.c : next_wave).



Étape 2 : 24 aliens avec 6 sprites

3. Le jeu complet

Projet de cette étape : **tutoriel/etape3**. On complète aliens.c (plongeurs, bombes, vaisseau mère), play.c (capsule, vaisseau touché), fx.c (explosions, kill_ship) et main.c (vies, deux joueurs, fin de partie). Le code est celui du jeu final ; la documentation du code le détaille fonction par fonction. Voici les idées principales.

3.1 Les plongeurs : une machine à états

Un plongeur passe par trois états : il décolle, il attaque (guidé vers le vaisseau, en lâchant des bombes), puis il revient se placer dans la formation. Un `switch` sur l'état est la façon la plus claire d'écrire ce genre de comportement. Pendant son vol, son bit est posé dans `diving` : la formation ne le dessine plus, mais garde sa place.

```
static void diver_one(diver_t *d, uint8_t spr)
{
    uint8_t y, v;
    switch (d->state) {
    case 0:
        gx_spr_hide(spr);
        return;
    case 1:                                     /* decollage */
        d->y -= 2;
        div_move_x(d);
        if (!--d->timer) d->state = 2;
        break;
    case 2:                                     /* attaque */
        v = ply.wave >> 2;
        y = (uint8_t)d->y + (v < 2 ? v : 2) + 2;
        d->y = y;
        if (y >= 200) {                         /* sorti en bas : reapparait en haut */
            d->state = 3;
            d->y = -24;
            break;
        }
        if (!(frame_count & 3)) {               /* guidage vers le vaisseau */
            if (ship_x - d->x < 0) {
                if (d->vx != -4) d->vx--;
            } else if (d->vx != 4) {
                d->vx++;
            }
        }
        div_move_x(d);
        if (!--d->bomb) {                       /* bombe */
            d->bomb = (rand8() & 31) + 30;
            if (!ship_dead && y < 140) spawn_bomb(d->x, y + 8);
        }
        if (!ship_dead && abs16((int16_t)y - SHIP_Y) < 14 && abs16(ship_x - d->x) < 24) {
            kill_ship();                        /* collision avec le vaisseau */
            d->state = 0;
            alien_destroy(d->row, d->col);
            gx_spr_hide(spr);
            return;
        }
        break;
    default:                                    /* 3 : retour vers la formation */
        d->x = fx + d->col * COL_PITCH;
        d->y += 2;
        if (d->y >= (uint8_t)(fy + d->row * ROW_PITCH)) {
            d->state = 0;                       /* arrive : reprend sa place */
            diving[d->row] &= ~bits[d->col];
            gx_spr_hide(spr);
            return;
        }
        break;
    }
    gx_spr_set(spr, d->x, d->y, F_DIVE_A + wave_type, ZOOM_X2Y1);
}
```

3.2 Le vaisseau mère

Toutes les 4 vagues (`(ply.wave & 3) == 0`), les sprites 0 à 3 prennent les 4 quarts du vaisseau mère, en zoom x4 / x2 : 128 x 64 pixels. La table raster n'a plus qu'une bande. Touché, il devient blanc pendant 3 images : la bande reçoit la palette pal_white4 au lieu de pal_boss.



Le vaisseau mère

3.3 Joueurs et vies

Chaque joueur a son bloc player_t (score, vies, vague, aliens restants, capsules, points de vie du vaisseau mère). Le jeu travaille sur une copie, `ply`. À la fin d'un tour, la copie est rangée et l'autre joueur reprend sa partie exactement où il l'avait laissée. En C, copier une structure s'écrit simplement `ply = pblk[cur_player];`.

```
static void game(void)
{
    player_init(&pblk[0]);
    player_init(&pblk[1]);
    if (nplayers != 2) pblk[1].over = 1;        /* joueur 2 absent */
    cur_player = 0;
    for (;;) {
        ply = pblk[cur_player];                 /* copie de travail */
        play_turn();
        if (!ply.lives) {
            ply.over = 1;
            update_hi();
            if (nplayers == 2) player_over_msg();
        }
        pblk[cur_player] = ply;
        if (nplayers == 2 && !pblk[cur_player ^ 1].over) {
            cur_player ^= 1;                    /* au tour de l'autre joueur */
        } else if (ply.over) {
            game_over();                        /* plus personne : fin de partie */
            return;
        }
    }
}
```

3.4 Mais le jeu ralentit…

Dans cette étape, les étoiles et les bandes de sprites sont encore en C : le projet définit `STARS_EN_C=1 BANDES_EN_C=1` (Projet > Propriétés). En jouant, on remarque que la formation et les étoiles saccadent par moments. Le jeu n'arrive plus à tout faire en une image. L'étape 4 explique comment le mesurer et le corriger.

4. Mesurer, puis accélérer en assembleur

4.1 Le profil

Lancer le jeu (F5), commencer une partie, puis **Débogage > Mesurer les performances** (Ctrl+F6). Pendant 5 secondes, le Caprice32 corrigé compte les cycles du Z80 passés à chaque adresse. Le studio les regroupe ensuite par fichier, par fonction et par ligne dans l'onglet **Profil**.

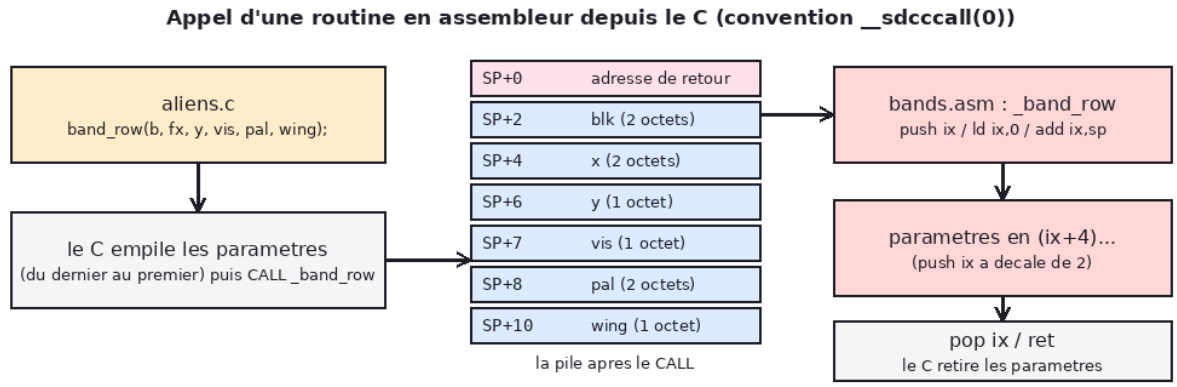
```
== Profil sur 5 s (250 images) : 20,004,084 cycles du Z80
```

```
   temps libre (attente de la fin d'image, wait_frame) : 0 %
   travail par image : 19,990 us en moyenne (une image dure 19 968 us)
   -- par fichier
    38.8 %  src/fx.c
    38.5 %  src/aliens.c
     6.7 %  lib/plus/sprites.asm
   -- routines (etiquettes) et fonctions qui prennent le plus de temps
    24.0 %  stars_update             src/fx.c:158
    13.9 %  band_row                 src/aliens.c:260
    13.9 %  star_ptr                 src/fx.c:152
     8.7 %  put_spr                  src/aliens.c:218
     4.8 %  put_colors               src/aliens.c:229
```

Le diagnostic est clair. Le jeu a besoin de **19 990 µs** par image : une image dure 19 968 µs, il ne reste aucun temps libre. Le jeu saute donc des images. Et **deux fonctions prennent les deux tiers du temps** : les étoiles (stars_update + star_ptr : 38 %) et le remplissage des bandes (band_row + put_spr + put_colors : 27 %). Le reste du jeu, collisions et plongeurs compris, compte peu.

4.2 band_row en assembleur

On crée **src/bands.asm** (Projet > Nouveau source…). Une fonction C devient une étiquette précédée de _ : `_band_row`. Avec `__sdcccall(0)` (déclaration dans stellar.h), les paramètres sont sur la pile, dans l'ordre. On les lit avec IX, en sauvant IX, que le C utilise pour ses variables locales.



Les paramètres de band_row sur la pile

```
_band_row:
        push ix
        ld ix,0
        add ix,sp                       ; (ix+4) = premier parametre
        ld l,(ix+4)
        ld h,(ix+5)                     ; HL = bloc
        ld e,(ix+6)
        ld d,(ix+7)                     ; DE = X
        ld c,(ix+9)                     ; C = aliens visibles
        ld b,6
bdr_spr:
        ld (hl),e                       ; X
        inc hl
        ld (hl),d
        inc hl
        ld a,(ix+8)                     ; Y
        ld (hl),a
        inc hl
        ld (hl),0
        inc hl
        xor a
        rr c
        jr nc,bdr_z
        ld a,ZOOM_X2Y1
bdr_z:  ld (hl),a                       ; zoom (0 = invisible)
        inc hl
        ld a,e                          ; colonne suivante : X + 48
        add a,48
```

```
        ld e,a
        jr nc,bdr_c
        inc d
bdr_c:  djnz bdr_spr
        ex de,hl                        ; DE = couleurs 1-4 du bloc
        ld l,(ix+10)
        ld h,(ix+11)
        ld bc,8
        ldir
        ex de,hl                        ; HL = fin du bloc
        ld a,(ix+12)
        or a
        jr z,bdr_w0
        dec hl                          ; ailes basses : couleur 3 au noir
        dec hl
        dec hl
        ld (hl),0
        dec hl
        ld (hl),0
        jr bdr_end
bdr_w0: dec hl                          ; ailes hautes : couleur 4 au noir
        ld (hl),0
        dec hl
        ld (hl),0
bdr_end:
        pop ix
        ret
```

4.3 Les étoiles en assembleur

De même, **src/stars.asm** contient `_stars_update`. La routine lit le tableau C `stars` (sous le nom `_stars`) et la variable `_star_skip`. Elle appelle la fonction C `rand8` (`call _rand8`, résultat dans L). Le tableau stars[] n'est plus `static` dans fx.c : un nom static n'est pas visible en dehors de son fichier.

```
_stars_update:
        ld hl,_stars                    ; HL parcourt le tableau (IX n'est pas utilise)
        ld b,ST_NB
st_loop:
        push bc
        ld c,(hl)                       ; C = colonne
        inc hl
        inc hl
        ld a,(hl)                       ; A = ligne
        call st_ptr                     ; DE = adresse de l'etoile a l'ecran
        jr z,st_1
        xor a
        ld (de),a                       ; efface
st_1:   ld a,(hl)
        inc hl
        add a,(hl)                      ; descend de sa vitesse
        dec hl
        cp 184
        jr c,st_2
        sub 168                         ; repart en haut...
        ld (hl),a
        dec hl
        dec hl
        push hl
        call st_col                     ; ...dans une colonne au hasard
        pop hl
        ld (hl),a
        ld c,a
        inc hl
        inc hl
        ld a,(hl)
st_2:   ld (hl),a
        call st_ptr
        jr z,st_3
        dec hl
        ld a,(hl)                       ; motif
        ld (de),a                       ; dessine
        inc hl
```

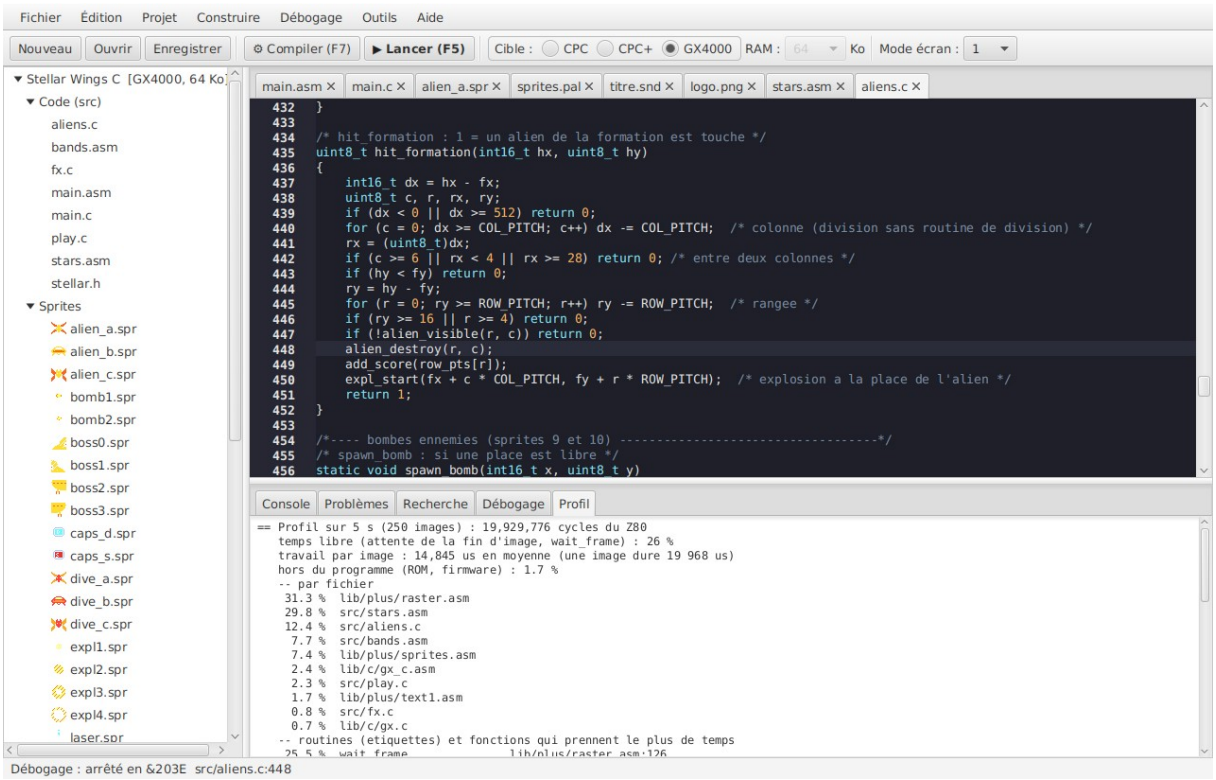
```
st_3:   inc hl                          ; etoile suivante
        inc hl
        pop bc
        djnz st_loop
        ret
```

4.4 Brancher les routines

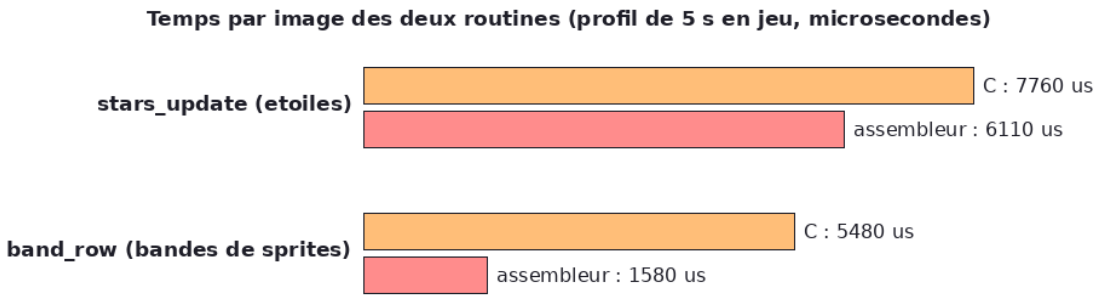
Dans **src/main.asm**, après la liaison avec la bibliothèque, on inclut les deux fichiers. Dans les fichiers C, on garde les versions C entre `#if STARS_EN_C` et `#if BANDES_EN_C` pour pouvoir comparer. Pour finir, on vide le champ Définitions de Projet > Propriétés.

```
        include "../lib/c/gx_c.asm"     ; liaison C <-> bibliotheque
        ifndef STARS_EN_C               ; (STARS_EN_C=1 : version C de fx.c, pour comparer)
        include "stars.asm"             ; etoiles du fond (appelees par le C)
        endif
        ifndef BANDES_EN_C              ; (BANDES_EN_C=1 : version C d'aliens.c)
        include "bands.asm"             ; bandes de sprites de la formation
        endif
```

4.5 Mesurer à nouveau



Le profil avec les deux routines en assembleur



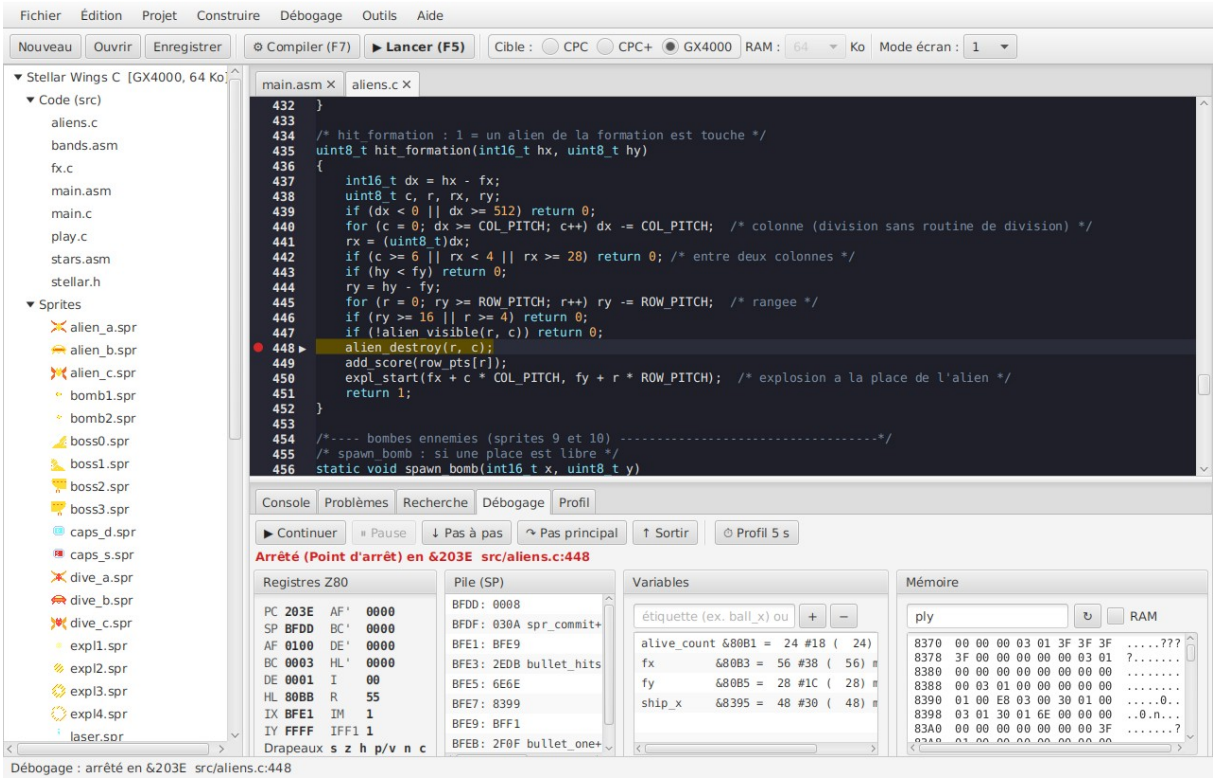
travail total par image : 19 990 us tout en C (plus qu'une image : le jeu ralentit), 15 870 us avec les 2 routines en assembleur (21 % de temps libre). Une image dure 19 968 us.

Avant / après

Le travail par image tombe à environ **15 900 µs** : il reste 20 à 30 % de temps libre selon les moments, et le jeu tient 50 images par seconde. band_row est 3,5 fois plus rapide. En assembleur, HL avance tout seul dans le bloc ; en C, le pointeur et les compteurs vivent dans la pile. Les étoiles ne gagnent que 25 % : leur coût vient surtout du calcul des adresses à l'écran, aussi long en assembleur. Pour aller plus loin, il faut un meilleur algorithme : garder l'adresse de chaque étoile pour l'effacer sans recalcul. À vous !

- Écrire tout le jeu en C, mesurer, puis réécrire seulement ce qui coûte : on garde un code lisible et un jeu rapide.
- Le profil montre aussi les fonctions de la bibliothèque de SDCC (divisions, multiplications) : à remplacer par des décalages ou des soustractions quand elles apparaissent en haut de la liste.
- Toujours garder une marge de temps libre : la console fait le même travail que l'émulateur, mais un moment chargé (explosions, plongeurs, texte) peut coûter plus que la moyenne.

5. Déboguer le C



Arrêt dans hit_formation quand un tir touche un alien

- **F9** sur une ligne C pose un point d'arrêt ; **F11** avance d'une ligne C, **F10** passe un appel, **Maj+F11** sort de la fonction, **F8** continue.
- **Variables** : le nom d'une variable globale ou static (alive_count, fx, ship_x). **Mémoire** : `ply` montre le joueur en cours, octet par octet.
- **Projet > Propriétés** : `START_WAVE=4` pour tester le vaisseau mère tout de suite, `GOD_MODE=1` pour un vaisseau invincible.
- Une erreur de SDCC s'affiche dans l'onglet Problèmes avec le fichier et la ligne (double-clic pour y aller).

6. Pour aller plus loin

- **Étoiles plus rapides** : ajouter l'adresse de dessin à star_t (6 octets), l'utiliser pour effacer ; adapter stars_init et stars.asm. Mesurer avant et après.

- **Une quatrième forme d'alien** : alien_d.spr, dive_d.spr, rowpal_tab, et le modulo de wave_prepare.
- **Un bonus de plus** : une capsule « bouclier » (type 2) qui protège d'un coup.
- **Le score sur la console** : copier build/stellarw.cpr sur la PicoGX, et comparer votre meilleur score à celui de la version assembleur !